LESSON 18

交副本 n = f(n) 接回来

函数参数与作用域

学完以后，你能说清函数里改了什么、外面为什么没变，以及怎样把结果拿回来。

函数里加了 1，n 怎么没变?

change.cpp

```cpp
int change(int x) {
    x++;
    return x;
}
int main() {
    int n = 0;
    cin >> n;
    change(n);
    cout << n << endl;
```

输入 4。change 里明明写了 x++。

屏幕上

4

输出还是 4，不是 5。
这节课就弄清楚为什么。

交给函数的，是一份副本

调用 change(n) 时，程序把 n 盒子里的数**抄一份**，放进函数自己的盒子 x。

n	x	x++
main 的盒子，里面是 4	change 的盒子，里面是抄来的 4	改的是 x，n 里还是 4

就像把作业复印一份交出去：别人在复印件上涂改，你手里的原件不变。

这节课说的 int 参数都是这样传的。

输入 4：两个盒子各变各的

第 12 行	`change(n), n 是 4`	4 抄进 x
第 5 行	`x++`	x 变成 5，n 还是 4
第 6 行	`return x`	交回 5，可是没人接
第 13 行	`cout << n`	输出 4

函数结束时，它自己的盒子 x 也跟着没了。

想让 n 变成 5，要把交回来的 5 放回 n 里。

想改 n，就把结果接回来

add_one.cpp

```
 4 int addOne(int x) {
 5     return x + 1;
 6 }
 8 int main() {
 9     int n = 0;
10     cin >> n;
11     n = addOne(n);
12     cout << n << endl;
```

第 11 行先算右边：addOne(n) 交回 5，再放进左边的 n。

和 n = n + 1 一样，只是加 1 这件事交给了函数。

输入 4

5

change.cpp

Ln 5

小测：只写 addOne(n);，输入 4 输出几?

1 4

2 5

3 报错

4 什么也没有

答对了！addOne 交回了 5，可是没有放回 n，n 还是 4。

动手改一改

从 add_one 开始。函数改成翻倍，再调用两次，每次都接回来。

翻倍两次

读入整数 n，是 -1000 到 1000。写函数 twice 返回 x 的两倍，用它让 n 翻倍两次，再输出 n。

输入 3

```
12
```

输入 -5

```
-20
```

名字一样，也是两个盒子

same.cpp

```cpp
int f() {
    int x = 100;
    return x;
}
int main() {
    int x = 1;
    f();
    cout << x << endl;
```

在大括号里准备的变量，只在这对大括号里能用，这叫它的**作用域**。

f 里的 x 和 main 里的 x 各有各的地盘，互不打扰。

猜一猜：same.cpp 输出几？

same.cpp

```
 9 int main() {
10     int x = 1;
11     f();
12     cout << x << endl;
```

屏幕上

1

main 里的 x 一直是 1。名字一样，不代表是同一个盒子。

小测：change 里能直接用 main 的 n 吗?

1 能，n 在程序里

2 不能，要用参数交进去

3 能，只要 n 写在前面

4 能，只要参数也叫 n

答对了！n 只在 main 里能用。要让函数用到它，就把它当参数交进去。

动手写一写

`bigger` 已经写好了。先比两个，再拿结果和第三个比。

三个数里最大的

一行读入三个整数，都是 `-1000` 到 `1000`，用空格隔开。调用 `bigger` 两次，输出最大的那个。

输入 `-4 -2 -8`

```
-2
```

输入 `2 3 1`

```
3
```

读报错：main 里找不到 x

same.cpp（只阅读）

```
 4 int f() {
 5     int x = 100;
 6     return x;
 7 }
 8 int main() {
10     f();
11     cout << x << endl;
```

```
11:13: error: 'x' was not declared in this scope
```

x 是在 f 的大括号里准备的，出了 f 就没了。main 里找不到它。

想用 f 算出来的数，就接住它：int x = f();

在线 IDE 显示的文字和位置可能不一样，还是先找行号和 error。

找错：哪一行有问题？

add_two.cpp（只阅读）

```cpp
int n = 0;
cin >> n;
n = addOne(n);
addOne(n);
cout << n << endl;
```

这段想让 n 加两次 1。
addOne 函数没有问题，这里没画出来。
输入 4，应该是 6，屏幕上却是 5。
点你认为有问题的那一行。

点代码里的一行

自己挑战

小树长高

一行读入树高 h（1 到 100）和天数 d（0 到 10）。小树每天长高自己的一半，按整数除法算：h + h / 2。写函数 grow 算出长一天以后的高度，用它算出 d 天后的高度。

输入 10 2，输出（10 → 15 → 22）

22

输入 10 0，输出

10

输入 1 5，输出

1

先自己写，卡住了再翻下一页。

卡住了？一次看一条提示

提示 1　grow 只管一天：交进去今天的高度，交回明天的高度。

提示 2　有几天就调用几次，用 for 来重复。

提示 3　每次都要把结果接回 h，否则下一天还是从原来的高度长。

一点点变化的小程序

写一个函数，让一个数变一点点；调用几次，每次都接回来。主题你自己定。

先做到

写一个有参数、有 `return` 的函数，用 `n = f(n)` 调用至少两次。

选你喜欢的

宠物喂食、手机充电、存钱罐加钱、游戏打怪掉血，都可以。

还想更厉害

用循环调用很多次，每次输出变化后的数。

没有标准答案。写之前说清想要的结果，写完试试最小和最大的输入。

和同桌换着玩

1

换着运行

运行同桌的程序，先猜调用三次后变成几

2

说一个喜欢的

告诉同桌，你最喜欢这个程序哪里

3

提一个想法

问问同桌：如果忘了接回来，会怎样?

听完再改一版。想一想、做出来、试一试、分享、再改，程序就是这样越做越好的。

说得清吗?

Q1 **函数里 x++，main 里的 n 为什么不变?**

交进去的是 n 的副本，改的是函数自己的 x。

Q2 **想让 n 变，要怎么写?**

把结果接回来：n = addOne(n);。

Q3 **f 里准备的变量，main 里能用吗?**

不能，它只在 f 的大括号里有效。

速查卡

```
n = addOne(n);
```

`int` 参数	交进去的是副本，改它不影响外面
`n = f(n)`	想让 `n` 变，就把结果接回来
`addOne(n);`	`addOne` 只交回结果，没人接，`n` 就不变
作用域	变量只在准备它的大括号里能用
同名变量	不同大括号里的同名变量，是不同的盒子

这些后面再学

引用 &

不交副本，直接把原来的盒子交进去

结构体

把几样东西打包成一条记录

递归

函数自己调用自己

今天学会了：函数拿到的是副本，想要结果就接回来。

谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 cpp-for-youth

下一课：结构体入门